

4.3 Hardware

A parte que forma o núcleo de Hardware no projeto do Micromouse é relacionado à percepção do mundo e controle de suas ações.

Foi escolhido para atuar como sistema central do robô o microcontrolador ESP32 DevKit V1. Inicialmente, houve dúvida entre ele e o Arduino Leonardo, porém optou-se por aquele pela sua capacidade de operar com frequências maiores (240MHz) e trabalho com 32 bits. Logo, é possível realizar mais operações se comparado ao Arduino em questão, que trabalha em (16 MHz) e 8 bits. Afinal, essas métricas afetam diretamente a capacidade do microcontrolador fazer cálculos complexos, enquanto o Arduino vai de 0 a 255 por vez, a ESP32 DevKit V1 processa palavras de 32 bits em um clock, assim, o código irá ser executado com uma rapidez consideravelmente maior, característica que se mostra fundamental no requisito que demanda a transmissão de dados de telemetria em tempo real via Wi-fi para um sistema web.

Além disso, o Micromouse precisa simultaneamente: enviar dados de telemetria pro sistema web e comunicar com os demais componentes, recebendo e enviando comandos para eles. Com isso em mente, a ESP32 DevKit V1 tem dois núcleos, o Application CPU e o Protocol CPU, este tem como função transmitir as informações para o website e aquele cuidar das demandas do mundo real. Em decorrer disso, a ESP32 DevKit V1 é capaz de separar as tarefas e fazê-las ao mesmo tempo e, por esse motivo, deve se mostrar mais adequada perante a necessidade de mapear o labirinto, andar nele e ter Wi-Fi e bluetooth integrados, facilitando a visualização do labirinto e seu mapeamento e evitando travar o funcionamento do próprio Micromouse ao mandar os dados por Wi-fi.

Os sensores têm como objetivo realizar a leitura do ambiente e, após pesquisar a fundo, foi decidido que o sensor infravermelho (IR) será uma opção mais coerente se. Projeto Conceitual do Produto 27 comparada ao sensor laser. Isso se deve ao fato de que o sensor IR tem uma latência menor de dados, assim, ao identificar uma parede, o microcontrolador será avisado o mais rápido possível e terá tempo de reagir adequadamente a essa informação. Além disso, devido ao ambiente de uso do Micromouse ser controlado e o labirinto não possuir variações de cores, o uso do sensor IR não apresenta nenhuma contraindicação. Mas para entender melhor o porquê disso é crucial compreender como o sensor IR funciona: ele atira uma 'luz invisível' e mede o quanto dessa luz é refletida de volta, desse modo, a monocromacia do labirinto permite que o programa de software possa seguir um padrão de lógica ao medir os valores entregues pelos sensores e se torne possível o cálculo da distância. Com isso, depois de escolhidos, é importante determinar quantos serão utilizados. Após discutir com a equipe de software, foi decidido que serão usados quatro sensores: dois frontais angulados, para ser capaz de identificar curvas de forma antecipada, e dois, para manter-se centralizado nos corredores do labirinto.

Para complementar o sistema de percepção, o RataTuring utiliza motores DC N20 equipados com encoders magnéticos embutidos. Mas por que esses sensores são importantes para o Micromouse? Ao imaginá-lo percorrendo um corredor longo, sem paredes na frente perto o suficiente para a leitura eficaz da luz refletida pelos sensores IR, fica evidente a necessidade de medir a distância percorrida por ele para ser possível mapear quantas células do labirinto passaram. Dessa maneira, os encoders leem a rotação do eixo do motor e geram pulsos elétricos a cada fração de giro, depois, o microcontrolador recebe e contabiliza esses pulsos digitais em tempo real. Assim, essa conversão direta de movimento

mecânico em dados digitais permite que o sistema calcule, com alta precisão, a velocidade atual das rodas e a distância exata percorrida, informações essenciais para o mapeamento e o rastreamento da posição do robô no labirinto

O próximo impasse foi como regular o nível de tensão da bateria de Polímero de Lítio (LiPo) que oferece 7.4V. Isso se mostra necessário pois a ESP32 DevKit V1 só suporta 5V como alimentação.

Inicialmente, foi cogitado o uso de dois resistores simples organizados num divisor de tensão para resolver esse problema de maneira simples e barata. Porém, o problema está justamente na matemática do divisor de tensão:

$$V_{out} = V_{in} \cdot \frac{R_2}{R_1 + R_2} \quad (4.3)$$

que só funciona perfeitamente quando nada está consumindo energia daquele fio do meio. Entretanto, o ESP 32 não é um componente passivo, ele é dinâmico. Quando ele está apenas usando o núcleo Application CPU, consome pouca energia (por volta de 40 miliamperes). Mas, no momento em que ele liga o Protocol CPU para enviar a telemetria do trajeto, ele já puxa um pico gigantesco de energia (até 400 miliamperes).

Na prática, quando o ESP 32 puxa essa corrente, ele atua como se fosse um terceiro resistor invisível que muda de valor o tempo todo. Dessa maneira, a voltagem que era 3.3V despenca instantaneamente para 1.5V ou menos. Assim, sem energia suficiente, provavelmente o microcontrolador vai apagar e reiniciar sozinho, ou seja, o micromouse morreria no meio do labirinto.

Em seguida, pensei em um diodo Zener (junto com um resistor) ou um regulador linear tradicional que conseguiriam manter a tensão estável em 3.3V mesmo quando o microcontrolador puxar muita energia. Porém, o problema deles é como eles fazem isso: eles transformam o excesso de energia em forma de calor. No RataTuring, houve o receio de que isso viraria um desastre por estar drenando e desperdiçando o consumo de bateria do Micromouse apenas para gerar calor no chassi do carrinho e fazendo com que a bateria durasse poucos minutos.

Finalmente, foi acertado o uso de um módulo Buck Converter LM2596 para baixar essa energia. Assim, um Buck Converter (ou conversor Step-Down) é uma válvula redutora de pressão inteligente. De maneira que, ao invés de simplesmente frear a energia e transformar o excesso em calor, como um resistor (o que desperdiçaria a bateria do Micromouse muito rápido), ele liga e desliga a energia milhares de vezes por segundo. Logo, isso reduz a voltagem média que sai do outro lado de forma extremamente eficiente, fazendo a bateria durar muito mais tempo.

No âmbito de bateria, o requisito do projeto onde se faz necessário haver a leitura do nível da bateria a todo momento em tempo real, fez crucial a implementação de um circuito divisor de tensão. Ele irá contar com dois resistores organizados de modo para que a ESP 32 consiga ler a voltagem da bateria sem queimar, cumprindo o requisito de enviar esse dado para a telemetria.

Em última instância, foi discutida a necessidade de uma Ponte H para fazer a comunicação entre os sinais da ESP32 DevKit V1 com os motores DC N20. Afinal, dessa maneira, os sinais da ESP32 DevKit V1 irão ser suficientemente ampliados, dado que ela funciona em uma voltagem muito menor do que os motores e, caso eles tentassem se

comunicar sem a ponte H, a ESP32 DevKit V1 poderia ser danificada por não ter o sinal suficiente para alimentá-los. Quanto ao modelo, a preferência foi do modelo TB6612FNG ao invés do modelo L298N, pois seu tamanho e sua resistência interna são bem menores. Logo, culminará em maior estabilidade de tensão, desse modo, assegurando confiabilidade maior, suportando frequências de até 100 kHz e permitindo o controle de velocidade sem vibração do motor ou perda de força em baixas velocidades.